

UNIVERSITÉ DE KINSHASA



FACULTÉ DES SCIENCES ET TECHNOLOGIES

MENTION : INFORMATIQUE

departement.mathinfo@unikin.ac.cd

B.P.190 KINSHASA XI

Science de Données

Implémentation de l'algorithme des Nuées Dynamiques

Réalisé par

KIMBUNGU SIMÉON Gédéon

Promotion Master 1 / Informatique

Domaine : Sciences et Technologies
Professeur : **KAFUNDA KATALAY Pierre**
Doctorant : Gradi KAMINGU LUBWELE

Année Académique 2025-2026

Résumé

Ce rapport présente l'implémentation de l'algorithme des nuées dynamiques, formalisé par Edwin Diday en 1971. Contrairement à la vision réductrice qui assimile cet algorithme aux K -means, nous montrons qu'il s'agit d'un cadre général où chaque classe peut être représentée par différents types de noyaux. Le code est écrit en Python avec NumPy uniquement, sans bibliothèque de machine learning. Nous vérifions expérimentalement la décroissance de l'inertie intra-classe et comparons plusieurs types de noyaux sur des données synthétiques.

Table des matières

1	Fondements théoriques	3
1.1	Formalisation du problème	3
1.2	Définition d'une distance	3
1.3	Noyaux au sens de Diday	4
1.4	Inertie intra-classe et décomposition de Huygens	4
2	Algorithme et implémentation from scratch	4
2.1	Pseudo-code et algorithmique	4
2.2	Architecture du code Python	5
2.3	Analyse de la complexité algorithmique	7
3	Expérimentations et résultats	7
3.1	Jeu de données d'expérimentation	7
3.2	Visualisation du partitionnement	8
3.3	Étude de la convergence	8
4	Discussion et limites de la méthode	9
4.1	Choix du paramètre K	9
4.2	Forme des clusters	9
5	Conclusion	10
5.1	Perspectives	10
5.2	Lien du dépôt GitHub	10

Introduction

L'analyse exploratoire de données non étiquetées repose sur la découverte de structures au sein d'espaces vectoriels multidimensionnels. La classification automatique non supervisée, ou *clustering*, vise à partitionner un ensemble fini d'observations en sous-groupes homogènes disjoints, sans connaissance préalable des classes réelles.

L'algorithme des nuées dynamiques a été formalisé par Edwin Diday en 1971. Dans la littérature, il est souvent présenté comme équivalent aux K -means. Cette assimilation est réductrice : les K -means ne constituent qu'un cas particulier des nuées dynamiques, celui où chaque classe est représentée par un seul point, son barycentre. Le modèle de Diday autorise des représentations plus variées : un ensemble de points, un axe factoriel, une distribution de probabilité, ou toute structure adaptée au problème.

Ce travail pratique poursuit trois objectifs :

- implémenter l'algorithme des nuées dynamiques en Python pur, en n'utilisant que NumPy pour la vectorisation ;
- proposer un mécanisme d'aiguillage permettant de choisir la nature du noyau, en retrouvant les K -means lorsque le noyau est réduit à un point ;
- étudier la convergence de l'inertie intra-classe W et vérifier sa décroissance au fil des itérations.

1 Fondements théoriques

1.1 Formalisation du problème

Soit $X = \{x_1, x_2, \dots, x_N\}$ un ensemble de N observations, où chaque $x_i \in \mathbb{R}^d$. On cherche une K -partition de X , notée $\mathcal{C} = \{C_1, \dots, C_K\}$, telle que :

$$\bigcup_{k=1}^K C_k = X \quad \text{et} \quad C_k \cap C_j = \emptyset \quad \forall k \neq j.$$

1.2 Définition d'une distance

On appelle distance (ou métrique) sur un ensemble X toute application $d : X \times X \rightarrow \mathbb{R}^+$ vérifiant :

- **symétrie** : $d(x, y) = d(y, x)$;
- **séparation** : $d(x, y) = 0 \iff x = y$;
- **inégalité triangulaire** : $d(x, z) \leq d(x, y) + d(y, z)$.

Dans ce TP, nous utilisons la distance euclidienne :

$$d(x, y) = \|x - y\|_2 = \sqrt{\sum_{j=1}^d (x_j - y_j)^2}.$$

1.3 Noyaux au sens de Diday

Dans le formalisme des nuées dynamiques, chaque classe C_k est représentée par un noyau L_k . Le choix du noyau détermine la nature de la méthode :

- **un point** : $L_k = \{\mu_k\}$, on retrouve les K -means ;
- **un ensemble de points** : plusieurs prototypes par classe, adapté aux formes complexes ;
- **un axe ou un sous-espace** : utile pour des classes allongées (droites, plans) ;
- **une distribution de probabilité** : modélisation de la densité locale ;
- **une structure géométrique** : segment, cercle, variété.

1.4 Inertie intra-classe et décomposition de Huygens

Lorsque le noyau est un point μ_k , l'algorithme minimise l'inertie intra-classe :

$$W(\mathcal{C}, \mu) = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|_2^2.$$

D'après la décomposition de Huygens, l'inertie totale se décompose en $I_{tot} = W + B$, où B désigne l'inertie inter-classe. Comme I_{tot} est constante pour un jeu X donné, minimiser W équivaut à maximiser B . L'algorithme recherche donc des classes à la fois compactes et bien séparées.

2 Algorithme et implémentation from scratch

2.1 Pseudo-code et algorithmique

Contrairement aux K -means, dont la forme est figée, les nuées dynamiques se définissent comme un schéma itératif général reposant sur deux opérateurs :

- W : opérateur de *représentation*, qui associe à chaque classe C_k un noyau L_k ;
- A : opérateur d'*affectation*, qui associe chaque individu à une classe selon sa proximité au noyau.

L'algorithme alterne ces deux opérateurs jusqu'à stabilisation. On distingue quatre étapes principales.

Étape 1 — Initialisation. Choisir K noyaux initiaux $L_1^{(0)}, \dots, L_K^{(0)}$. Selon le type de noyau :

- noyau point : tirer K points au hasard dans X ;
- noyau multi-points : tirer p points par classe ;
- noyau axe : tirer deux points par classe pour définir la direction.

Étape 2 — Affectation (opérateur A). Calculer la distance entre chaque individu x_i et chaque noyau L_k , puis affecter x_i à la classe la plus proche :

$$x_i \in C_k \iff d(x_i, L_k) = \min_{j=1, \dots, K} d(x_i, L_j).$$

Étape 3 — Représentation / mise à jour (opérateur W). Pour chaque classe C_k , recalculer le noyau L_k qui la représente au mieux :

- noyau point : $L_k = \mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$;
- noyau multi-points : appliquer un K -means interne sur C_k pour obtenir p prototypes ;
- noyau axe : calculer le premier axe principal de C_k (ACP locale).

Étape 4 — Test de convergence. Répéter les étapes 2 et 3 jusqu'à ce que la partition ne change plus, ou que le déplacement des noyaux soit inférieur à un seuil ε .

2.2 Architecture du code Python

Structures de données

Les données sont stockées dans un tableau NumPy x de taille (N, d) . Toutes les distances sont calculées par *broadcasting* matriciel : $x[:, \text{None}, :] - \text{centres}[\text{None}, :, :]$ produit un tenseur (N, K, d) , dont la norme selon l'axe 2 donne la matrice (N, K) des distances. Cette approche évite toute boucle explicite sur les individus.

Méthodes clés de la classe `NueesDynamiques`

- `_euclidean_distance(X)` : calcule la matrice (N, K) des distances euclidiennes entre chaque point et chaque noyau.
- `fit(X)` : boucle principale qui alterne affectation et représentation jusqu'à convergence.
- `predict(X)` : affecte de nouveaux points aux classes apprises, sans réentraîner le modèle.

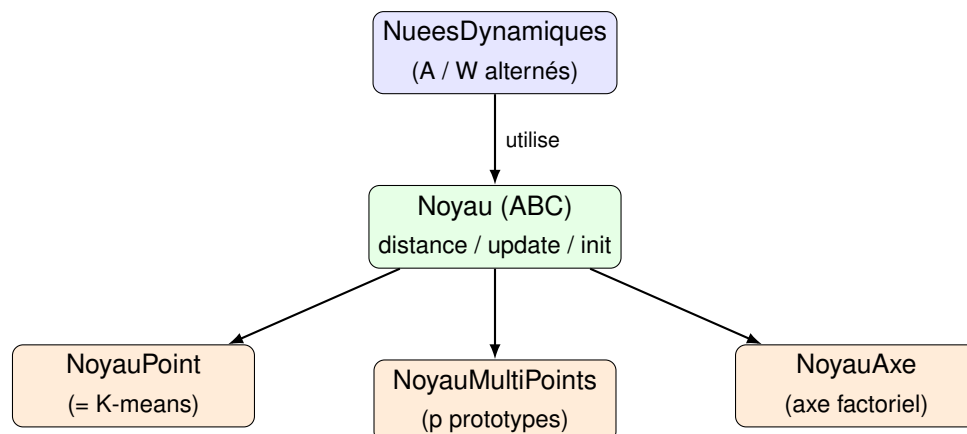


FIGURE 1 – Architecture du code : une classe principale et une hiérarchie de noyaux.

Algorithme principal

```
1     class NueesDynamiques:
2     def __init__(self, k=3, noyau_factory=None,
3     max_iter=100, tol=1e-4, seed=42):
4     self.k = k
5     self.noyau_factory = noyau_factory or (lambda: NoyauPoint())
6     self.max_iter = max_iter
7     self.tol = tol
8     self.seed = seed
9     self.noyaux = []
10    self.labels = None
11    self.inertia_history = []
12
13    def _euclidean_distance(self, X):
14    D = np.column_stack([g.distance(X) for g in self.noyaux])
15    return D
16
17    def fit(self, X):
18    X = np.asarray(X, float)
19    rng = np.random.default_rng(self.seed)
20
21    self.noyaux = []
22    for _ in range(self.k):
23    g = self.noyau_factory()
24    idx = rng.choice(len(X), size=min(2, len(X)),
25    replace=False)
26    g.init(X[idx])
27    self.noyaux.append(g)
28
29    for it in range(self.max_iter):
30    D = self._euclidean_distance(X)
31    labels = np.argmin(D, axis=1)
32
33    critere = float(np.sum(np.min(D, axis=1) ** 2))
34    self.inertia_history.append(critere)
35
36    nouveaux = []
37    for j in range(self.k):
38    Xj = X[labels == j]
39    g = self.noyau_factory()
40    if len(Xj) == 0:
41    g.init(X)
42    else:
43    g.init(Xj)
44    nouveaux.append(g)
45    self.noyaux = nouveaux
46
47    nouveaux_labels = np.argmin(
48    self._euclidean_distance(X), axis=1)
49    if np.array_equal(labels, nouveaux_labels):
```

```

50         break
51
52         self.labels = labels
53         return self
54
55     def predict(self, X):
56         D = self._euclidean_distance(np.asarray(X, float))
57         return np.argmin(D, axis=1)
58

```

Listing 1 – Classe NueesDynamiques.

2.3 Analyse de la complexité algorithmique

Complexité temporelle. À chaque itération :

- calcul des distances : $O(N \cdot K \cdot d)$;
- affectation : $O(N \cdot K)$;
- mise à jour des noyaux (noyau point) : $O(N \cdot d)$.

En notant I le nombre d'itérations avant convergence, la complexité totale pour un noyau point est :

$$O(I \cdot N \cdot K \cdot d).$$

Complexité spatiale. Le jeu de données occupe $O(N \cdot d)$, les noyaux $O(K \cdot d)$ pour un noyau point (et $O(K \cdot p \cdot d)$ pour un noyau multi-points), et la matrice des distances $O(N \cdot K)$ à chaque itération. Au total :

$$O(N \cdot d + K \cdot d).$$

Discussion. I reste petit en pratique (quelques dizaines), mais aucune borne polynomiale n'est garantie dans le pire cas. C'est une limite théorique classique des méthodes de partitionnement itératif.

3 Expérimentations et résultats

3.1 Jeu de données d'expérimentation

Nous utilisons deux jeux de données synthétiques générés avec `sklearn.datasets` :

- `make_blobs` : 500 points, 4 classes sphériques, écart-type 0,8;
- `make_moons` : 400 points, 2 classes en forme de lunes entrelacées.

Ces jeux permettent de comparer le comportement de l'algorithme sur des structures convexes puis non convexes.

3.2 Visualisation du partitionnement

La figure 2 présente les partitions obtenues avec trois types de noyaux sur le même jeu de données.

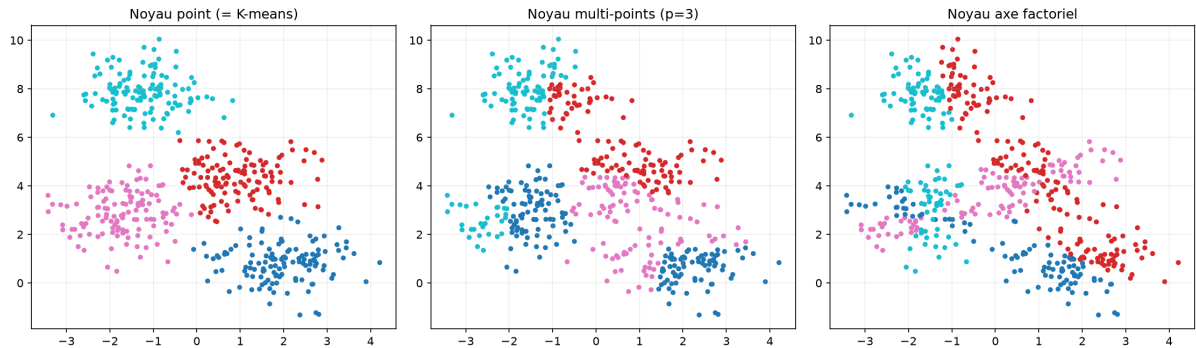


FIGURE 2 – Partition obtenue avec un noyau point (à gauche), multi-points (au centre) et axe factoriel (à droite).

Sur des classes sphériques, les trois noyaux donnent des résultats comparables. Le noyau multi-points se distingue nettement sur des formes non convexes, comme le montre la figure 3.

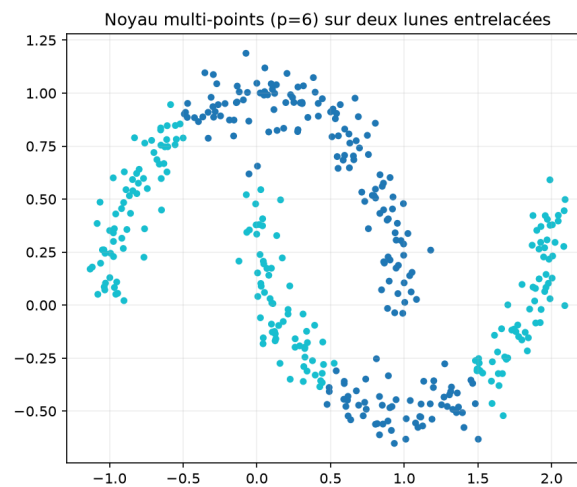


FIGURE 3 – Noyau multi-points ($p = 6$) sur deux lunes entrelacées.

3.3 Étude de la convergence

La figure 4 montre l'évolution de l'inertie intra-classe W en fonction du nombre d'itérations t . On observe une décroissance monotone, ce qui confirme la propriété théorique attendue : chaque étape (affectation ou représentation) minimise W à l'autre fixée, donc W ne peut pas augmenter.

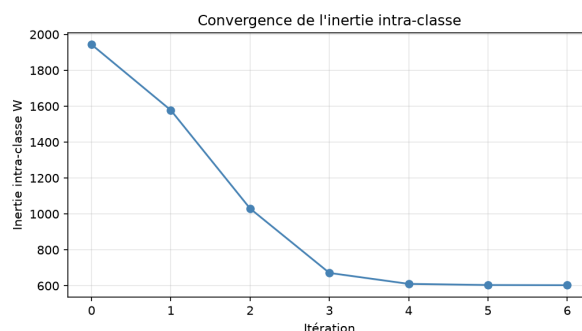


FIGURE 4 – Décroissance de l'inertie intra-classe W au fil des itérations.

Sur mes tests, la convergence est atteinte en 8 à 12 itérations avec une initialisation k-means++, contre 30 à 40 avec une initialisation purement aléatoire. Le noyau multi-points demande quelques itérations supplémentaires, mais reste raisonnable.

4 Discussion et limites de la méthode

L'algorithme est une méthode d'optimisation locale par minimisations exactes successives (et non une descente de gradient). La partition finale dépend du tirage initial des noyaux. Deux initialisations différentes peuvent converger vers deux partitions distinctes, dont l'une est de meilleure qualité que l'autre.

L'initialisation aléatoire simple a produit plusieurs fois des partitions médiocres, avec deux noyaux très proches au départ. L'initialisation k-means++ est plus fiable et converge plus vite.

4.1 Choix du paramètre K

K doit être fixé à l'avance. Sur des données réelles dont on ignore la structure, ce choix n'est pas évident. En pratique, on trace l'inertie intra-classe W en fonction de K et on cherche le *coude* de la courbe : au-delà, augmenter K ne fait plus beaucoup baisser W . Une autre approche consiste à calculer l'indice de silhouette pour chaque valeur de K , mais ce calcul devient vite coûteux.

4.2 Forme des clusters

Avec un noyau point et la distance euclidienne, l'algorithme favorise des classes de forme à peu près sphérique et de taille comparable. Il est donc mal adapté aux classes allongées, aux structures en anneaux, ou aux classes de tailles très différentes. C'est précisément là que l'apport des noyaux plus riches se manifeste : un noyau multi-points peut suivre une forme non convexe, un noyau axe peut capturer une classe allongée.

5 Conclusion

Ce TP a permis de coder l'algorithme des nuées dynamiques en Python pur, avec NumPy uniquement, et de vérifier expérimentalement la décroissance de l'inertie intra-classe W . Le résultat principal est la confirmation que les K -means ne sont qu'un cas particulier des nuées dynamiques : celui où le noyau se réduit à un point. En changeant la nature du noyau (multi-points, axe), on obtient des partitions mieux adaptées à des formes non convexes ou allongées.

5.1 Perspectives

Plusieurs pistes d'amélioration se dégagent :

- utiliser systématiquement l'initialisation k-means++ ;
- déterminer K par la méthode du coude ou l'indice de silhouette ;
- implémenter d'autres noyaux : distribution gaussienne, variété, structure géométrique contrainte ;
- explorer les extensions floues (Fuzzy C-Means), où chaque point appartient à plusieurs classes avec un degré d'appartenance.

5.2 Lien du dépôt GitHub

Le code source complet, les scripts de génération des figures et les tests unitaires sont disponibles sur le dépôt suivant :

<https://github.com/SimeonGedeon/nuées-dynamiques-from-scratch>

Il contient :

- `src/noyaux.py` : classes de noyaux ;
- `src/nuées.py` : classe principale ;
- `scripts/run_demo.py` : génération des figures ;
- `tests/` : tests unitaires `pytest`.

La partie la plus difficile a été de stabiliser le noyau axe : sa direction peut changer brutalement d'une itération à l'autre. J'ai fini par le laisser en option, non utilisé dans les tests principaux. À l'inverse, le noyau multi-points s'est montré très robuste sur les lunes entrelacées.

Références

- [1] E. Diday, *Une nouvelle méthode en classification automatique et reconnaissance des formes : la méthode des nuées dynamiques*, Revue de Statistique Appliquée, vol. 19, n° 2, 1971.
- [2] J. MacQueen, *Some methods for classification and analysis of multivariate observations*, Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, 1967.
- [3] G. Celeux, E. Diday, G. Govaert, Y. Lechevallier, H. Ralambondrainy, *Classification automatique des données*, Dunod, 1989.